

hwk2/control.vhd

```

1  -----
2  --
3  --  SH-2 Control Unit
4  --
5  --  This is an implementation of the control unit for the SH-2 it contains a
6  --  state machine to control the execution of instructions and generate the
7  --  appropriate control signals for the other blocks, the ALU, PAU, DAU, and
8  --  the register array. It also contains the logic to handle interrupts and
9  --  exceptions.
10 --
11 --  Entities included are:
12 --      ControlSH2 - the control unit
13 --
14 --  Revision History:
15 --      20 Apr 26  Simone Shevchuk      Initial revision.
16 --      26 Apr 26  Simone Shevchuk      Add control signals for all instructions, add default values for signals,
17 --                                     no FSM
18 --      28 Apr 26  Simone Shevchuk      reg_ax_in removed
19 --      29 Apr 26  Simone Shevchuk      Add clock handling for memory access/write
20 --      01 May 26  Simone Shevchuk      Double instruction fetch mux IR
21 --      03 May 26  Simone Shevchuk      Update DAU controls for MOV instructions
22 --      05 May 26  Simone Shevchuk      Add imm_sel control signal for selecting type of immediate value and add
23 --                                     control signals for NEG/NEGC
24 --      08 Apr 26  Simone Shevchuk      Add delay slot handling, add comments
25 --      11 May 26  Simone Shevchuk      Change register select in byte word MOV for register indirect
26 --      12 May 26  Simone Shevchuk      Add alu_opb_bit7 output for TAS.B bit 7 set
27 --      12 May 26  Simone Shevchuk      Fix STC.L GBR/VBR src_sel, add dau_load_data
28 -----
29 --
30 --  ControlSH2
31 --
32 --  This is the control unit for the SH-2 processor. It contains a state machine
33 --  to control the execution of instructions and generate the appropriate control
34 --  signals for the other blocks, the ALU, PAU, DAU, and the register array.
35 --  It also contains the logic to handle interrupts and exceptions.
36 --
37 --  Inputs:
38 --      DB              - value coming in from the DB
39 --      Tbit            - Tbit used for conditional branches
40 --      clock           - the system clock
41 --      reset           - reset signal
42 --      addr_bit1       - bit 1 of the address for determining whether to load upper or lower half-word into IR
43 --
44 --  Outputs:
45 --      reg_ALU_IN      - the control signals for register array that feed into ALU/back into register array for
46 --                        updates
47 --      reg_MAU         - the control signals for the register array that feed into the PAU and DAU
48 --      alu_IN          - the control signals for the ALU source selection
49 --      alu_gen         - the control signals for the generic ALU, check documentation for details on what each bit
50 --                        does
51 --      alu_sh2         - the control signals for the ALU signals specific to SH2 operations, like shifts and extends
52 --      dau_controls    - the control signals for the dau controls like source selection, offset selection, and
53 --                        inc/dec
54 --      dau_write       - the control signals for writing to the GBR and VBR from the DAU
55 --      dau_out_sel     - the control signals for whether to output addr_src_out, GBR, VBR from DAU
56 --      pau_controls    - the control signals for the PAU controls like source selection and PR writing
57 --      RE              - the control signals for the read enables for memory access
58 --      WE              - the control signals for the write enables for memory access
59 --      addr_bus_sel    - the control signal to select the address bus source (PAU or DAU)
60 --      delay_slot      - signal to indicate whether the current instruction is in a delay slot for branch delay
61 --                        handling
62 --      data_bus_sel    - the control signal to select for data bus source, 0 for ALU, 1 for DAU (GBR or VBR), 2 for
63 --                        SR in CPU, 3 for PR

```

```

59  -- sign_zero_ALU_sel  - the control signal to select for whether to sign extend or zero extend immediate values
   for ALU input
60  -- tbit_write        - the control signal to select whether to write to the T bit for conditional instructions
   that set T bit based on their result
61  -- IR                - the latched instruction register
62  -- imm_sel           - the control signal to select for which type of immediate value to use, 00=8b sign, 01=4b
   zero, 10=12b sign, 11=8b zero
63  -- alu_opa_zero      - the control signal to select whether to force ALU operand A to zero for instructions like
   NEG/NEGC
64  -- alu_opa_hold      - the control signal to select whether to use the DB hold register as ALU operand A instead
   of the live DB for read-modify-write instructions
65  -- alu_opb_bit7      - the control signal to select whether to force ALU operand B to 0x00000080 for TAS.B bit 7
   set
66  -- dau_load_data     - the control signal to select whether to load GBR/VBR from the data bus instead of the ALU
   output for instructions like LDC.L
67
68  library ieee;
69  use ieee.std_logic_1164.all;
70  use ieee.numeric_std.std_match;
71  use work.InstructionConstants.all;
72
73  entity ControlSH2 is
74
75      port(
76          DB                : in  std_logic_vector(31 downto 0);
77          addr_bit1         : in  std_logic; -- bit 1 of the address for determining whether to load upper or lower
   half-word into IR
78          Tbit              : in  std_logic;
79          clock              : in  std_logic;
80          reset              : in  std_logic;
81          reg_ALU_IN        : out  std_logic_vector(7 downto 0); -- [3(reg_in_src) + 4(reg_in_sel) + 1(reg_store)]
82          reg_MAU           : out  std_logic_vector(16 downto 0); -- [4(reg_a_sel) + 4(reg_b_sel) + 4(reg_ax_sel) +
   1(reg_ax_store) + 4(reg_a2_sel)]
83          alu_IN            : out  std_logic_vector(1 downto 0); -- [1(alu_sel_a) + 1(alu_sel_b)]
84          alu_gen           : out  std_logic_vector(10 downto 0); -- [4(f_cmd) + 2(c_in_cmd) + 3(s_cmd) + 2(alu_cmd)]
85          alu_sh2           : out  std_logic_vector(10 downto 0); -- [2(shift_sel) + 2(extend_sel) + 3(res_sel) +
   4(t_sel)]
86          dau_controls      : out  std_logic_vector(9 downto 0); -- [2(dau_src_sel) + 2(dau_shift_sel) +
   2(dau_offset_sel) + 1(dau_inc_dec_sel) + 2(dau_inc_dec_bit) + 1(dau_pre_post_sel)]
87          dau_write         : out  std_logic_vector(1 downto 0); -- [1(gbr_write) + 1(vbr_write)]
88          dau_out_sel       : out  std_logic_vector(1 downto 0); -- [2(data_out_sel)]
89          pau_controls      : out  std_logic_vector(5 downto 0); -- [2(pau_src_sel) + 2(pau_offset_sel) + 2(pr_sel)]
90          RE                : buffer std_logic_vector(3 downto 0); -- [4(read enable for byte/halfword/word accesses)]
91          WE                : out  std_logic_vector(3 downto 0); -- [4(write enable for byte/halfword/word accesses)]
92          addr_bus_sel      : buffer std_logic;
93          delay_slot        : buffer std_logic;
94          data_bus_sel      : out  std_logic_vector(1 downto 0);
95          sign_zero_ALU_sel : out  std_logic;
96          IR                : buffer std_logic_vector(15 downto 0); -- latched instruction register
97          tbit_write        : out  std_logic; -- signal to select whether to write to the T bit for conditional
   instructions that set T bit based on their result
98          imm_sel           : out  std_logic_vector(1 downto 0); -- 00=8b sign, 01=4b zero, 10=12b sign, 11=8b zero
99          alu_opa_zero      : out  std_logic; -- force ALU operand A to zero (for NEG/NEGC)
100         alu_opa_hold       : out  std_logic; -- select DB hold register as ALU operand A instead of live DB (for
   read-modify-write instructions)
101         alu_opb_bit7       : out  std_logic; -- force ALU operand B to 0x00000080 (for TAS.B bit 7 set)
102         dau_load_data      : out  std_logic -- select DB as source for GBR/VBR writes in DAU (for LDC.L)
103     );
104
105 end ControlSH2;
106
107
108 architecture structural of ControlSH2 is
109     signal cycleCnt        : integer range 0 to 7 := 0; -- counter for the current cycle of instruction
   execution
110     signal last_cycle      : std_logic := '0'; -- signal to indicate the last cycle of instruction execution
111     signal delay_slot_reg  : std_logic := '0'; -- registered delay_slot to break combinational loop
112

```

```

113     alias mreg                : std_logic_vector(3 downto 0) is IR(7 downto 4); -- alias types since just need to
reference the bits in the IR
114     alias nreg                : std_logic_vector(3 downto 0) is IR(11 downto 8);
115 begin
116
117
118
119     process(clock)
120     begin
121         if rising_edge(clock) then
122             if last_cycle = '1' then
123                 cycleCnt <= 0;
124             else
125                 cycleCnt <= cycleCnt + 1;
126             end if;
127             delay_slot_reg <= delay_slot;
128         end if;
129     end process;
130
131     process(clock)
132     begin
133         if rising_edge(clock) then
134             if reset = '0' then
135                 IR <= (others => '0');
136             elsif RE = "0000" and addr_bus_sel = '0' then
137                 if addr_bit1 = '0' then
138                     IR <= DB(31 downto 16); -- upper half-word
139                 else
140                     IR <= DB(15 downto 0); -- lower half-word
141                 end if;
142             end if;
143         end if;
144     end process;
145
146     process(all)
147     begin
148
149         -- Combinational defaults -- overridden only where an instruction needs a different value
150         reg_ALU_IN      <= REG_IN_NONE;
151         reg_MAU         <= REG_MAU_NONE;
152         alu_IN          <= ALU_IN_DATAREG;
153         alu_gen         <= ALU_GEN_PASS;
154         dau_out_sel     <= "00"; -- addr_src_out (overridden for STC GBR/VBR)
155         data_bus_sel    <= "00"; -- ALU output (overridden for DAU/SR/PR sources)
156         dau_controls    <= DAU_CTRL_NONE;
157         dau_write       <= DAU_WRITE_NONE;
158         alu_sh2         <= ALU_SH2_NONE;
159         pau_controls    <= PAU_SEQ;
160         RE              <= RE_LONG;
161         WE              <= WE_NONE;
162         addr_bus_sel    <= '0'; -- PAU drives address bus
163         delay_slot      <= '0';
164         sign_zero_ALU_sel <= '0';
165         last_cycle      <= '1'; -- single-cycle by default; multi-cycle instructions override
166         tbit_write      <= '0';
167         imm_sel         <= "00"; -- 8-bit sign-extended by default
168         alu_opa_zero     <= '0'; -- only set for NEG/NEGC to zero ALU operand A
169         alu_opa_hold     <= '0';
170         alu_opb_bit7     <= '0';
171         dau_load_data    <= '0';
172
173         if std_match(IR, RESET_VECTOR_AT_0) then -- This is not an actual instruction, but a pattern
174             -- to match for the reset vector fetch to set the PC to 0
175             pau_controls <= PAU_HOLD;
176
177
178         elsif std_match(IR, MOV_IMM_IMM_RN) then -- MOV #imm,Rn

```

```

179     reg_ALU_IN      <= ("000" & nreg & "1");
180     alu_IN          <= ALU_IN_DATAIMM;
181     alu_gen         <= ALU_GEN_IMM;
182     sign_zero_ALU_sel <= '1'; -- sign extend immediate
183
184     elsif std_match(IR, MOV_RM_RN) then -- MOV Rm,Rn
185         reg_ALU_IN      <= ("000" & nreg & "1");
186         reg_MAU         <= (mreg & "000000000000");
187         alu_IN          <= ALU_IN_REGREG;
188         alu_gen         <= ALU_GEN_PASS;
189
190     elsif std_match(IR, MOV_W_AT_DISP_PC_RN) then -- MOV.W @(disp,PC),Rn
191         imm_sel         <= "11";
192         if cycleCnt = 0 then
193             reg_ALU_IN      <= ("001" & nreg & "1");
194             alu_IN          <= ALU_IN_DATAREG;
195             alu_gen         <= ALU_GEN_IMM;
196             dau_controls    <= "1101000000";
197             pau_controls    <= "001000";
198             RE             <= RE_WORD;
199             addr_bus_sel    <= '1';
200             last_cycle      <= '0';
201         elsif cycleCnt = 1 then
202             reg_ALU_IN      <= ("001" & nreg & "0");
203             alu_IN          <= ALU_IN_DATAREG;
204             alu_gen         <= ALU_GEN_IMM;
205             dau_controls    <= "1101000000";
206         end if;
207
208     elsif std_match(IR, MOV_L_AT_DISP_PC_RN) then -- MOV.L @(disp,PC),Rn
209         imm_sel         <= "11";
210         if cycleCnt = 0 then
211             reg_ALU_IN      <= ("001" & nreg & "1");
212             alu_IN          <= ALU_IN_DATAREG;
213             alu_gen         <= ALU_GEN_IMM;
214             dau_controls    <= "1110000000";
215             pau_controls    <= "001000";
216             addr_bus_sel    <= '1';
217             last_cycle      <= '0';
218         elsif cycleCnt = 1 then
219             reg_ALU_IN      <= ("001" & nreg & "0");
220             alu_IN          <= ALU_IN_DATAREG;
221             alu_gen         <= ALU_GEN_IMM;
222             dau_controls    <= "1110000000";
223         end if;
224
225     elsif std_match(IR, MOV_B_RM_AT_RN) then -- MOV.B Rm,@Rn
226         if cycleCnt = 0 then
227             reg_MAU         <= (mreg & "000000000" & nreg);
228             alu_IN          <= ALU_IN_REGREG;
229             alu_gen         <= ALU_GEN_PASS;
230             pau_controls    <= "001000";
231             RE             <= RE_NONE;
232             WE             <= WE_BYTE;
233             addr_bus_sel    <= '1';
234             last_cycle      <= '0';
235         elsif cycleCnt = 1 then
236             reg_MAU         <= (mreg & "000000000" & nreg);
237             alu_IN          <= ALU_IN_REGREG;
238             alu_gen         <= ALU_GEN_PASS;
239         end if;
240
241     elsif std_match(IR, MOV_W_RM_AT_RN) then -- MOV.W Rm,@Rn
242         if cycleCnt = 0 then
243             reg_MAU         <= (mreg & "000000000" & nreg);
244             alu_IN          <= ALU_IN_REGREG;
245             alu_gen         <= ALU_GEN_PASS;

```

```

246         pau_controls      <= "001000";
247         RE                 <= RE_NONE;
248         WE                 <= WE_WORD;
249         addr_bus_sel       <= '1';
250         last_cycle         <= '0';
251     elsif cycleCnt = 1 then
252         reg_MAU             <= (mreg & "000000000" & nreg);
253         alu_IN              <= ALU_IN_REGREG;
254         alu_gen             <= ALU_GEN_PASS;
255     end if;
256
257     elsif std_match(IR, MOV_L_RM_AT_RN) then -- MOV.L Rm,@Rn
258         if cycleCnt = 0 then
259             reg_MAU          <= (mreg & "000000000" & nreg);
260             alu_IN           <= ALU_IN_REGREG;
261             alu_gen          <= ALU_GEN_PASS;
262             pau_controls     <= "001000";
263             RE               <= RE_NONE;
264             WE               <= WE_LONG;
265             addr_bus_sel     <= '1';
266             last_cycle       <= '0';
267         elsif cycleCnt = 1 then
268             reg_MAU          <= (mreg & "000000000" & nreg);
269             alu_IN           <= ALU_IN_REGREG;
270             alu_gen          <= ALU_GEN_PASS;
271         end if;
272
273     elsif std_match(IR, MOV_B_AT_RM_RN) then -- MOV.B @Rm,Rn
274         if cycleCnt = 0 then
275             reg_ALU_IN       <= ("001" & nreg & "1");
276             reg_MAU          <= ("0000000000000" & mreg);
277             alu_IN           <= ALU_IN_DATAREG;
278             alu_gen          <= ALU_GEN_PASS;
279             pau_controls     <= "001000";
280             RE               <= RE_BYTE;
281             addr_bus_sel     <= '1';
282             last_cycle       <= '0';
283         elsif cycleCnt = 1 then
284             reg_ALU_IN       <= ("001" & nreg & "0");
285             reg_MAU          <= ("0000000000000" & mreg);
286             alu_IN           <= ALU_IN_DATAREG;
287             alu_gen          <= ALU_GEN_PASS;
288         end if;
289
290     elsif std_match(IR, MOV_W_AT_RM_RN) then -- MOV.W @Rm,Rn
291         if cycleCnt = 0 then
292             reg_ALU_IN       <= ("001" & nreg & "1");
293             reg_MAU          <= ("0000000000000" & mreg);
294             alu_IN           <= ALU_IN_DATAREG;
295             alu_gen          <= ALU_GEN_PASS;
296             pau_controls     <= "001000";
297             RE               <= RE_WORD;
298             addr_bus_sel     <= '1';
299             last_cycle       <= '0';
300         elsif cycleCnt = 1 then
301             reg_ALU_IN       <= ("001" & nreg & "0");
302             reg_MAU          <= ("0000000000000" & mreg);
303             alu_IN           <= ALU_IN_DATAREG;
304             alu_gen          <= ALU_GEN_PASS;
305         end if;
306
307     elsif std_match(IR, MOV_L_AT_RM_RN) then -- MOV.L @Rm,Rn
308         if cycleCnt = 0 then
309             reg_ALU_IN       <= ("001" & nreg & "1");
310             reg_MAU          <= ("0000000000000" & mreg);
311             alu_IN           <= ALU_IN_DATAREG;
312             alu_gen          <= ALU_GEN_PASS;

```

```

313         pau_controls      <= "001000";
314         addr_bus_sel      <= '1';
315         last_cycle        <= '0';
316     elsif cycleCnt = 1 then
317         reg_ALU_IN         <= ("001" & nreg & "0");
318         reg_MAU            <= ("000000000000" & mreg);
319         alu_IN             <= ALU_IN_DATAREG;
320         alu_gen            <= ALU_GEN_PASS;
321     end if;
322
323     elsif std_match(IR, MOV_B_AT_RM_PLUS_RN) then -- MOV.B @Rm+,Rn
324         if cycleCnt = 0 then
325             reg_ALU_IN      <= ("001" & nreg & "1");
326             reg_MAU         <= ("00000000" & mreg & "1" & mreg);
327             alu_IN          <= ALU_IN_DATAREG;
328             alu_gen         <= ALU_GEN_PASS;
329             dau_controls    <= "0000100000";
330             pau_controls    <= "001000";
331             RE              <= RE_BYTE;
332             addr_bus_sel    <= '1';
333             last_cycle      <= '0';
334         elsif cycleCnt = 1 then
335             reg_ALU_IN      <= ("001" & nreg & "0");
336             alu_IN          <= ALU_IN_DATAREG;
337             alu_gen         <= ALU_GEN_PASS;
338         end if;
339
340     elsif std_match(IR, MOV_W_AT_RM_PLUS_RN) then -- MOV.W @Rm+,Rn
341         if cycleCnt = 0 then
342             reg_ALU_IN      <= ("001" & nreg & "1");
343             reg_MAU         <= ("00000000" & mreg & "1" & mreg);
344             alu_IN          <= ALU_IN_DATAREG;
345             alu_gen         <= ALU_GEN_PASS;
346             dau_controls    <= "0000100010";
347             pau_controls    <= "001000";
348             RE              <= RE_WORD;
349             addr_bus_sel    <= '1';
350             last_cycle      <= '0';
351         elsif cycleCnt = 1 then
352             reg_ALU_IN      <= ("001" & nreg & "0");
353             alu_IN          <= ALU_IN_DATAREG;
354             alu_gen         <= ALU_GEN_PASS;
355         end if;
356
357     elsif std_match(IR, MOV_L_AT_RM_PLUS_RN) then -- MOV.L @Rm+,Rn
358         if cycleCnt = 0 then
359             reg_ALU_IN      <= ("001" & nreg & "1");
360             reg_MAU         <= ("00000000" & mreg & "1" & mreg);
361             alu_IN          <= ALU_IN_DATAREG;
362             alu_gen         <= ALU_GEN_IMM;
363             dau_controls    <= "0000100100";
364             pau_controls    <= "001000";
365             addr_bus_sel    <= '1';
366             last_cycle      <= '0';
367         elsif cycleCnt = 1 then
368             reg_ALU_IN      <= ("001" & nreg & "0");
369             alu_IN          <= ALU_IN_DATAREG;
370             alu_gen         <= ALU_GEN_IMM;
371         end if;
372
373     elsif std_match(IR, MOV_B_RM_AT_MINUSRN) then -- MOV.B Rm,@-Rn
374         if cycleCnt = 0 then
375             reg_MAU         <= (mreg & "0000" & nreg & "1" & nreg);
376             alu_IN          <= ALU_IN_REGREG;
377             alu_gen         <= ALU_GEN_PASS;
378             dau_controls    <= "0000101001";
379             pau_controls    <= "001000";

```

```

380         RE                <= RE_NONE;
381         WE                <= WE_BYTE;
382         addr_bus_sel      <= '1';
383         last_cycle        <= '0';
384     elsif cycleCnt = 1 then
385         alu_IN            <= ALU_IN_REGREG;
386         alu_gen           <= ALU_GEN_PASS;
387     end if;
388
389     elsif std_match(IR, MOV_W_RM_AT_MINUSRN) then -- MOV.W Rm,@-Rn
390         if cycleCnt = 0 then
391             reg_MAU        <= (mreg & "0000" & nreg & "1" & nreg);
392             alu_IN         <= ALU_IN_REGREG;
393             alu_gen        <= ALU_GEN_PASS;
394             dau_controls    <= "0000101011";
395             pau_controls    <= "001000";
396             RE             <= RE_NONE;
397             WE             <= WE_WORD;
398             addr_bus_sel   <= '1';
399             last_cycle      <= '0';
400         elsif cycleCnt = 1 then
401             alu_IN         <= ALU_IN_REGREG;
402             alu_gen        <= ALU_GEN_PASS;
403         end if;
404
405     elsif std_match(IR, MOV_L_RM_AT_MINUSRN) then -- MOV.L Rm,@-Rn
406         if cycleCnt = 0 then
407             reg_MAU        <= (mreg & "0000" & nreg & "1" & nreg);
408             alu_IN         <= ALU_IN_REGREG;
409             alu_gen        <= ALU_GEN_PASS;
410             dau_controls    <= "0000101101";
411             pau_controls    <= "001000";
412             RE             <= RE_NONE;
413             WE             <= WE_LONG;
414             addr_bus_sel   <= '1';
415             last_cycle      <= '0';
416         elsif cycleCnt = 1 then
417             alu_IN         <= ALU_IN_REGREG;
418             alu_gen        <= ALU_GEN_PASS;
419         end if;
420
421     elsif std_match(IR, MOV_B_R0_AT_DISP_RN) then -- MOV.B R0,@(disp,Rn)
422         imm_sel            <= "01";
423         if cycleCnt = 0 then
424             reg_MAU        <= ("000000000000" & IR(7 downto 4));
425             alu_IN         <= ALU_IN_REGREG;
426             alu_gen        <= ALU_GEN_PASS;
427             pau_controls    <= "001000";
428             dau_controls    <= DAU_CTRL_OFFSET_DISP8;
429             RE             <= RE_NONE;
430             WE             <= WE_BYTE;
431             addr_bus_sel   <= '1';
432             last_cycle      <= '0';
433         elsif cycleCnt = 1 then
434             alu_IN         <= ALU_IN_REGREG;
435             alu_gen        <= ALU_GEN_PASS;
436         end if;
437
438     elsif std_match(IR, MOV_W_R0_AT_DISP_RN) then -- MOV.W R0,@(disp,Rn)
439         imm_sel            <= "01";
440         if cycleCnt = 0 then
441             reg_MAU        <= ("000000000000" & IR(7 downto 4));
442             alu_IN         <= ALU_IN_REGREG;
443             alu_gen        <= ALU_GEN_PASS;
444             dau_controls    <= "0001000000";
445             pau_controls    <= "001000";
446             RE             <= RE_NONE;

```

```

447         WE                                <= WE_WORD;
448         addr_bus_sel                       <= '1';
449         last_cycle                         <= '0';
450     elsif cycleCnt = 1 then
451         alu_IN                             <= ALU_IN_REGREG;
452         alu_gen                            <= ALU_GEN_PASS;
453     end if;
454
455     elsif std_match(IR, MOV_L_RM_AT_DISP_RN) then -- MOV.L Rm,@(disp,Rn)
456         imm_sel                             <= "01";
457         if cycleCnt = 0 then
458             reg_MAU                         <= (mreg & "00000000" & nreg);
459             alu_IN                          <= ALU_IN_REGREG;
460             alu_gen                         <= ALU_GEN_PASS;
461             dau_controls                    <= "001000000";
462             pau_controls                    <= "001000";
463             RE                              <= RE_NONE;
464             WE                              <= WE_LONG;
465             addr_bus_sel                    <= '1';
466             last_cycle                      <= '0';
467         elsif cycleCnt = 1 then
468             alu_IN                          <= ALU_IN_REGREG;
469             alu_gen                         <= ALU_GEN_PASS;
470         end if;
471
472     elsif std_match(IR, MOV_B_AT_DISP_RM_R0) then -- MOV.B @(disp,Rm),R0
473         imm_sel                             <= "01";
474         if cycleCnt = 0 then
475             reg_ALU_IN                      <= REG_IN_R0_ONLY;
476             reg_MAU                         <= ("000000000000" & mreg);
477             alu_IN                          <= ALU_IN_DATAREG;
478             alu_gen                         <= ALU_GEN_PASS;
479             pau_controls                    <= "001000";
480             dau_controls                    <= DAU_CTRL_OFFSET_DISP8;
481             RE                              <= RE_BYTE;
482             addr_bus_sel                    <= '1';
483             last_cycle                      <= '0';
484         elsif cycleCnt = 1 then
485             alu_IN                          <= ALU_IN_DATAREG;
486             alu_gen                         <= ALU_GEN_PASS;
487         end if;
488
489     elsif std_match(IR, MOV_W_AT_DISP_RM_R0) then -- MOV.W @(disp,Rm),R0
490         imm_sel                             <= "01";
491         if cycleCnt = 0 then
492             reg_ALU_IN                      <= REG_IN_R0_ONLY;
493             reg_MAU                         <= ("000000000000" & mreg);
494             alu_IN                          <= ALU_IN_DATAREG;
495             alu_gen                         <= ALU_GEN_PASS;
496             dau_controls                    <= "0001000000";
497             pau_controls                    <= "001000";
498             RE                              <= RE_WORD;
499             addr_bus_sel                    <= '1';
500             last_cycle                      <= '0';
501         elsif cycleCnt = 1 then
502             alu_IN                          <= ALU_IN_DATAREG;
503             alu_gen                         <= ALU_GEN_PASS;
504         end if;
505
506     elsif std_match(IR, MOV_L_AT_DISP_RM_RN) then -- MOV.L @(disp,Rm),Rn
507         imm_sel                             <= "01";
508         if cycleCnt = 0 then
509             reg_ALU_IN                      <= ("001" & nreg & "1");
510             reg_MAU                         <= ("000000000000" & mreg);
511             alu_IN                          <= ALU_IN_DATAREG;
512             alu_gen                         <= ALU_GEN_PASS;
513             dau_controls                    <= "001000000";

```

```

514         pau_controls      <= "001000";
515         addr_bus_sel      <= '1';
516         last_cycle        <= '0';
517     elsif cycleCnt = 1 then
518         reg_ALU_IN        <= ("001" & nreg & "0");
519         alu_IN             <= ALU_IN_DATAREG;
520         alu_gen            <= ALU_GEN_PASS;
521     end if;
522
523     elsif std_match(IR, MOV_B_RM_AT_R0_RN) then -- MOV.B Rm,@(R0,Rn)
524         if cycleCnt = 0 then
525             reg_MAU        <= (mreg & "000000000" & nreg);
526             alu_IN         <= ALU_IN_REGREG;
527             alu_gen        <= ALU_GEN_PASS;
528             dau_controls    <= "0000010000";
529             pau_controls    <= "001000";
530             RE              <= RE_NONE;
531             WE              <= WE_BYTE;
532             addr_bus_sel    <= '1';
533             last_cycle      <= '0';
534         elsif cycleCnt = 1 then
535             alu_IN          <= ALU_IN_REGREG;
536             alu_gen         <= ALU_GEN_PASS;
537         end if;
538
539     elsif std_match(IR, MOV_W_RM_AT_R0_RN) then -- MOV.W Rm,@(R0,Rn)
540         if cycleCnt = 0 then
541             reg_MAU        <= (mreg & "000000000" & nreg);
542             alu_IN         <= ALU_IN_REGREG;
543             alu_gen        <= ALU_GEN_PASS;
544             dau_controls    <= "0000010000";
545             pau_controls    <= "001000";
546             RE              <= RE_NONE;
547             WE              <= WE_WORD;
548             addr_bus_sel    <= '1';
549             last_cycle      <= '0';
550         elsif cycleCnt = 1 then
551             alu_IN          <= ALU_IN_REGREG;
552             alu_gen         <= ALU_GEN_PASS;
553         end if;
554
555     elsif std_match(IR, MOV_L_RM_AT_R0_RN) then -- MOV.L Rm,@(R0,Rn)
556         if cycleCnt = 0 then
557             reg_MAU        <= (mreg & "000000000" & nreg);
558             alu_IN         <= ALU_IN_REGREG;
559             alu_gen        <= ALU_GEN_PASS;
560             dau_controls    <= "0000010000";
561             pau_controls    <= "001000";
562             RE              <= RE_NONE;
563             WE              <= WE_LONG;
564             addr_bus_sel    <= '1';
565             last_cycle      <= '0';
566         elsif cycleCnt = 1 then
567             alu_IN          <= ALU_IN_REGREG;
568             alu_gen         <= ALU_GEN_PASS;
569         end if;
570
571     elsif std_match(IR, MOV_B_AT_R0_RM_RN) then -- MOV.B @(R0,Rm),Rn
572         if cycleCnt = 0 then
573             reg_ALU_IN      <= ("001" & nreg & "1");
574             reg_MAU         <= ("0000000000000" & mreg);
575             alu_IN          <= ALU_IN_DATAREG;
576             alu_gen         <= ALU_GEN_PASS;
577             dau_controls    <= "0000010000";
578             pau_controls    <= "001000";
579             RE              <= RE_BYTE;
580             addr_bus_sel    <= '1';

```

```

581         last_cycle           <= '0';
582     elsif cycleCnt = 1 then
583         reg_ALU_IN           <= ("001" & nreg & "0");
584         alu_IN               <= ALU_IN_DATAREG;
585         alu_gen              <= ALU_GEN_PASS;
586     end if;
587
588     elsif std_match(IR, MOV_W_AT_R0_RM_RN) then -- MOV.W @(R0,Rm),Rn
589         if cycleCnt = 0 then
590             reg_ALU_IN        <= ("001" & nreg & "1");
591             reg_MAU           <= ("000000000000" & mreg);
592             alu_IN            <= ALU_IN_DATAREG;
593             alu_gen           <= ALU_GEN_PASS;
594             dau_controls       <= "0000010000";
595             pau_controls       <= "001000";
596             RE                 <= RE_WORD;
597             addr_bus_sel      <= '1';
598             last_cycle        <= '0';
599             elsif cycleCnt = 1 then
600                 reg_ALU_IN    <= ("001" & nreg & "0");
601                 alu_IN        <= ALU_IN_DATAREG;
602                 alu_gen       <= ALU_GEN_PASS;
603                 dau_controls   <= "0000010000";
604             end if;
605
606         elsif std_match(IR, MOV_L_AT_R0_RM_RN) then -- MOV.L @(R0,Rm),Rn
607             if cycleCnt = 0 then
608                 reg_ALU_IN    <= ("001" & nreg & "1");
609                 reg_MAU       <= ("000000000000" & mreg);
610                 alu_IN        <= ALU_IN_DATAREG;
611                 alu_gen       <= ALU_GEN_PASS;
612                 dau_controls   <= "0000010000";
613                 pau_controls   <= "001000";
614                 addr_bus_sel   <= '1';
615                 last_cycle     <= '0';
616                 elsif cycleCnt = 1 then
617                     reg_ALU_IN <= ("001" & nreg & "0");
618                     alu_IN    <= ALU_IN_DATAREG;
619                     alu_gen   <= ALU_GEN_PASS;
620                 end if;
621
622             elsif std_match(IR, MOV_B_R0_AT_DISP_GBR) then -- MOV.B R0,@(disp,GBR)
623                 imm_sel      <= "11";
624                 if cycleCnt = 0 then
625                     alu_IN    <= ALU_IN_REGREG;
626                     alu_gen   <= ALU_GEN_PASS;
627                     dau_controls <= "0100000000";
628                     pau_controls <= "001000";
629                     RE        <= RE_NONE;
630                     WE        <= WE_BYTE;
631                     addr_bus_sel <= '1';
632                     last_cycle <= '0';
633                     elsif cycleCnt = 1 then
634                         alu_IN    <= ALU_IN_REGREG;
635                         alu_gen   <= ALU_GEN_PASS;
636                     end if;
637
638                 elsif std_match(IR, MOV_W_R0_AT_DISP_GBR) then -- MOV.W R0,@(disp,GBR)
639                     imm_sel      <= "11";
640                     if cycleCnt = 0 then
641                         alu_IN    <= ALU_IN_REGREG;
642                         alu_gen   <= ALU_GEN_PASS;
643                         dau_controls <= "0100000000";
644                         pau_controls <= "001000";
645                         RE        <= RE_NONE;
646                         WE        <= WE_WORD;
647                         addr_bus_sel <= '1';

```

```

648         last_cycle           <= '0';
649     elsif cycleCnt = 1 then
650         alu_IN                <= ALU_IN_REGREG;
651         alu_gen                <= ALU_GEN_PASS;
652     end if;
653
654     elsif std_match(IR, MOV_L_R0_AT_DISP_GBR) then -- MOV.L R0,@(disp,GBR)
655         imm_sel                <= "11";
656         if cycleCnt = 0 then
657             alu_IN              <= ALU_IN_REGREG;
658             alu_gen              <= ALU_GEN_PASS;
659             dau_controls         <= "0110000000";
660             pau_controls         <= "001000";
661             RE                   <= RE_NONE;
662             WE                   <= WE_LONG;
663             addr_bus_sel         <= '1';
664             last_cycle           <= '0';
665         elsif cycleCnt = 1 then
666             alu_IN              <= ALU_IN_REGREG;
667             alu_gen              <= ALU_GEN_PASS;
668         end if;
669
670     elsif std_match(IR, MOV_B_AT_DISP_GBR_R0) then -- MOV.B @(disp,GBR),R0
671         imm_sel                <= "11";
672         if cycleCnt = 0 then
673             reg_ALU_IN           <= REG_IN_R0_ONLY;
674             alu_IN               <= ALU_IN_DATAREG;
675             alu_gen              <= ALU_GEN_PASS;
676             dau_controls         <= "0100000000";
677             pau_controls         <= "001000";
678             RE                   <= RE_BYTE;
679             addr_bus_sel         <= '1';
680             last_cycle           <= '0';
681         elsif cycleCnt = 1 then
682             alu_IN              <= ALU_IN_DATAREG;
683             alu_gen              <= ALU_GEN_PASS;
684         end if;
685
686     elsif std_match(IR, MOV_W_AT_DISP_GBR_R0) then -- MOV.W @(disp,GBR),R0
687         imm_sel                <= "11";
688         if cycleCnt = 0 then
689             reg_ALU_IN           <= REG_IN_R0_ONLY;
690             alu_IN               <= ALU_IN_DATAREG;
691             alu_gen              <= ALU_GEN_PASS;
692             dau_controls         <= "0101000000";
693             pau_controls         <= "001000";
694             RE                   <= RE_WORD;
695             addr_bus_sel         <= '1';
696             last_cycle           <= '0';
697         elsif cycleCnt = 1 then
698             alu_IN              <= ALU_IN_DATAREG;
699             alu_gen              <= ALU_GEN_PASS;
700         end if;
701
702     elsif std_match(IR, MOV_L_AT_DISP_GBR_R0) then -- MOV.L @(disp,GBR),R0
703         imm_sel                <= "11";
704         if cycleCnt = 0 then
705             reg_ALU_IN           <= REG_IN_R0_ONLY;
706             alu_IN               <= ALU_IN_DATAREG;
707             alu_gen              <= ALU_GEN_PASS;
708             dau_controls         <= "0110000000";
709             pau_controls         <= "001000";
710             addr_bus_sel         <= '1';
711             last_cycle           <= '0';
712         elsif cycleCnt = 1 then
713             alu_IN              <= ALU_IN_DATAREG;
714             alu_gen              <= ALU_GEN_PASS;

```

```

715         end if;
716
717     elsif std_match(IR, MOVA_AT_DISP_PC_R0) then -- MOVA @(disp,PC),R0
718         imm_sel         <= "11";
719         reg_ALU_IN       <= "10000001";
720         alu_IN           <= ALU_IN_DATAREG;
721         alu_gen          <= ALU_GEN_IMM;
722         dau_controls     <= "1110000000";
723         dau_out_sel      <= "11";
724
725     elsif std_match(IR, MOVT_RN) then -- MOVT Rn
726         reg_ALU_IN       <= ("010" & nreg & "1");
727         alu_IN           <= ALU_IN_DATAREG;
728         alu_gen          <= ALU_GEN_NONE;
729
730     elsif std_match(IR, SWAP_B_RM_RN) then -- SWAP.B Rm,Rn
731         reg_ALU_IN       <= ("000" & nreg & "1");
732         reg_MAU          <= (mreg & "000000000000");
733         alu_IN           <= ALU_IN_REGREG;
734         alu_gen          <= ALU_GEN_PASS;
735         alu_sh2          <= "00000100000";
736
737     elsif std_match(IR, SWAP_W_RM_RN) then -- SWAP.W Rm,Rn
738         reg_ALU_IN       <= ("000" & nreg & "1");
739         reg_MAU          <= (mreg & "000000000000");
740         alu_IN           <= ALU_IN_REGREG;
741         alu_gen          <= "11000000110";
742
743     elsif std_match(IR, XTRCT_RM_RN) then -- XTRCT Rm,Rn
744         reg_ALU_IN       <= ("000" & nreg & "1");
745         reg_MAU          <= (nreg & mreg & "000000000");
746         alu_IN           <= ALU_IN_REGREG;
747         alu_gen          <= ALU_GEN_PASS;
748         alu_sh2          <= "00001000000";
749
750     elsif std_match(IR, ADD_RM_RN) then -- ADD Rm,Rn
751         reg_ALU_IN       <= ("000" & nreg & "1");
752         reg_MAU          <= (nreg & mreg & "000000000");
753         alu_IN           <= ALU_IN_REGREG;
754         alu_gen          <= ALU_GEN_ADD;
755
756     elsif std_match(IR, ADD_IMM_IMM_RN) then -- ADD #imm,Rn
757         reg_ALU_IN       <= ("000" & nreg & "1");
758         reg_MAU          <= (nreg & "000000000000");
759         alu_IN           <= ALU_IN_REGIMM;
760         alu_gen          <= ALU_GEN_ADD;
761         sign_zero_ALU_sel <= '1';
762
763     elsif std_match(IR, ADDC_RM_RN) then -- ADDC Rm,Rn
764         reg_ALU_IN       <= ("000" & nreg & "1");
765         reg_MAU          <= (nreg & mreg & "000000000");
766         alu_IN           <= ALU_IN_REGREG;
767         alu_gen          <= ALU_GEN_ADDC;
768         alu_sh2          <= "00000000001";
769         tbit_write       <= '1';
770
771     elsif std_match(IR, ADDV_RM_RN) then -- ADDV Rm,Rn
772         reg_ALU_IN       <= ("000" & nreg & "1");
773         reg_MAU          <= (nreg & mreg & "000000000");
774         alu_IN           <= ALU_IN_REGREG;
775         alu_gen          <= ALU_GEN_ADD;
776         alu_sh2          <= "00000000010";
777         tbit_write       <= '1';
778
779     elsif std_match(IR, CMP_EQ_RM_RN) then -- CMP/EQ Rm,Rn
780         reg_MAU          <= (nreg & mreg & "000000000");
781         alu_IN           <= ALU_IN_REGREG;

```

```

782         alu_gen          <= ALU_GEN_SUB;
783         alu_sh2           <= "00000000011";
784         tbit_write        <= '1';
785
786     elsif std_match(IR, CMP_EQ_IMM_IMM_R0) then -- CMP/EQ #imm,R0
787         alu_IN             <= ALU_IN_REGIMM;
788         alu_gen            <= ALU_GEN_SUB;
789         alu_sh2            <= "00000000011";
790         sign_zero_ALU_sel  <= '1';
791         tbit_write         <= '1';
792
793     elsif std_match(IR, CMP_GE_RM_RN) then -- CMP/GE Rm,Rn
794         reg_MAU            <= (nreg & mreg & "000000000");
795         alu_IN             <= ALU_IN_REGREG;
796         alu_gen            <= ALU_GEN_SUB;
797         alu_sh2            <= "00000000110";
798         tbit_write         <= '1';
799
800     elsif std_match(IR, CMP_GT_RM_RN) then -- CMP/GT Rm,Rn
801         reg_MAU            <= (nreg & mreg & "000000000");
802         alu_IN             <= ALU_IN_REGREG;
803         alu_gen            <= ALU_GEN_SUB;
804         alu_sh2            <= "00000000111";
805         tbit_write         <= '1';
806
807     elsif std_match(IR, CMP_HI_RM_RN) then -- CMP/HI Rm,Rn
808         reg_MAU            <= (nreg & mreg & "000000000");
809         alu_IN             <= ALU_IN_REGREG;
810         alu_gen            <= ALU_GEN_SUB;
811         alu_sh2            <= "00000000101";
812         tbit_write         <= '1';
813
814     elsif std_match(IR, CMP_HS_RM_RN) then -- CMP/HS Rm,Rn
815         reg_MAU            <= (nreg & mreg & "000000000");
816         alu_IN             <= ALU_IN_REGREG;
817         alu_gen            <= ALU_GEN_SUB;
818         alu_sh2            <= "00000000001";
819         tbit_write         <= '1';
820
821     elsif std_match(IR, CMP_PL_RN) then -- CMP/PL Rn
822         reg_MAU            <= (nreg & "0000000000000");
823         alu_IN             <= ALU_IN_REGREG;
824         alu_gen            <= ALU_GEN_PASS;
825         alu_sh2            <= "00000001000";
826         tbit_write         <= '1';
827
828     elsif std_match(IR, CMP_PZ_RN) then -- CMP/PZ Rn
829         reg_MAU            <= (nreg & "0000000000000");
830         alu_IN             <= ALU_IN_REGREG;
831         alu_gen            <= ALU_GEN_PASS;
832         alu_sh2            <= "00000001001";
833         tbit_write         <= '1';
834
835     elsif std_match(IR, CMP_STR_RM_RN) then -- CMP/STR Rm,Rn
836         reg_MAU            <= (nreg & mreg & "000000000");
837         alu_IN             <= ALU_IN_REGREG;
838         alu_gen            <= ALU_GEN_XOR;
839         alu_sh2            <= "00000001011";
840         tbit_write         <= '1';
841
842     elsif std_match(IR, EXTS_B_RM_RN) then -- EXTS.B Rm,Rn
843         reg_ALU_IN         <= ("000" & nreg & "1");
844         reg_MAU            <= (mreg & "0000000000000");
845         alu_IN             <= ALU_IN_REGREG;
846         alu_gen            <= ALU_GEN_PASS;
847         alu_sh2            <= "00000110000";
848

```

```

849     elsif std_match(IR, EXTS_W_RM_RN) then -- EXTS.W Rm,Rn
850         reg_ALU_IN      <= ("000" & nreg & "1");
851         reg_MAU         <= (mreg & "000000000000");
852         alu_IN          <= ALU_IN_REGREG;
853         alu_gen         <= ALU_GEN_PASS;
854         alu_sh2         <= "00010110000";
855
856     elsif std_match(IR, EXTU_B_RM_RN) then -- EXTU.B Rm,Rn
857         reg_ALU_IN      <= ("000" & nreg & "1");
858         reg_MAU         <= (mreg & "000000000000");
859         alu_IN          <= ALU_IN_REGREG;
860         alu_gen         <= ALU_GEN_PASS;
861         alu_sh2         <= "00100110000";
862
863     elsif std_match(IR, EXTU_W_RM_RN) then -- EXTU.W Rm,Rn
864         reg_ALU_IN      <= ("000" & nreg & "1");
865         reg_MAU         <= (mreg & "000000000000");
866         alu_IN          <= ALU_IN_REGREG;
867         alu_gen         <= ALU_GEN_PASS;
868         alu_sh2         <= "00110110000";
869
870     elsif std_match(IR, SUB_RM_RN) then -- SUB Rm,Rn
871         reg_ALU_IN      <= ("000" & nreg & "1");
872         reg_MAU         <= (nreg & mreg & "000000000");
873         alu_IN          <= ALU_IN_REGREG;
874         alu_gen         <= ALU_GEN_SUB;
875
876     elsif std_match(IR, SUBC_RM_RN) then -- SUBC Rm,Rn
877         reg_ALU_IN      <= ("000" & nreg & "1");
878         reg_MAU         <= (nreg & mreg & "000000000");
879         alu_IN          <= ALU_IN_REGREG;
880         alu_gen         <= ALU_GEN_SUBC;
881         alu_sh2         <= "00000001100";
882         tbit_write      <= '1';
883
884     elsif std_match(IR, SUBV_RM_RN) then -- SUBV Rm,Rn
885         reg_ALU_IN      <= ("000" & nreg & "1");
886         reg_MAU         <= (nreg & mreg & "000000000");
887         alu_IN          <= ALU_IN_REGREG;
888         alu_gen         <= ALU_GEN_SUBV;
889         alu_sh2         <= "00000000010";
890         tbit_write      <= '1';
891
892     elsif std_match(IR, NEG_RM_RN) then -- NEG Rm,Rn
893         reg_ALU_IN      <= ("000" & nreg & "1");
894         reg_MAU         <= ("0000" & mreg & "000000000");
895         alu_IN          <= ALU_IN_REGREG;
896         alu_gen         <= ALU_GEN_NEG;
897         alu_opa_zero    <= '1';
898
899     elsif std_match(IR, NEGC_RM_RN) then -- NEGC Rm,Rn
900         reg_ALU_IN      <= ("000" & nreg & "1");
901         reg_MAU         <= ("0000" & mreg & "000000000");
902         alu_IN          <= ALU_IN_REGREG;
903         alu_gen         <= ALU_GEN_NEGC;
904         alu_sh2         <= "00000001100";
905         alu_opa_zero    <= '1';
906
907     elsif std_match(IR, DT_RN) then -- DT Rn
908         reg_ALU_IN      <= ("000" & nreg & "1");
909         reg_MAU         <= (nreg & "000000000000");
910         alu_IN          <= ALU_IN_REGIMM;
911         alu_gen         <= "11110000001";
912         alu_sh2         <= "00000000011";
913         tbit_write      <= '1';
914
915     elsif std_match(IR, AND_RM_RN) then -- AND Rm,Rn

```

```

916     reg_ALU_IN      <= ("000" & nreg & "1");
917     reg_MAU         <= (nreg & mreg & "00000000");
918     alu_IN          <= ALU_IN_REGREG;
919     alu_gen         <= ALU_GEN_AND;
920
921     elsif std_match(IR, AND_IMM_IMM_R0) then -- AND #imm,R0
922         reg_ALU_IN      <= REG_IN_R0_REGFILE;
923         alu_IN          <= ALU_IN_REGIMM;
924         alu_gen         <= ALU_GEN_AND;
925
926     elsif std_match(IR, AND_B_IMM_IMM_AT_R0_GBR) then -- AND.B #imm,@(R0,GBR)
927         if cycleCnt = 0 then
928             alu_IN      <= ALU_IN_DATAIMM;
929             alu_gen      <= ALU_GEN_AND;
930             dau_controls <= "0100010000";
931             pau_controls <= "001000";
932             RE           <= RE_BYTE;
933             addr_bus_sel <= '1';
934             last_cycle   <= '0';
935             elsif cycleCnt = 1 then
936                 alu_IN      <= ALU_IN_DATAIMM;
937                 alu_gen      <= ALU_GEN_AND;
938                 alu_opa_hold <= '1';
939                 dau_controls <= "0100010000";
940                 pau_controls <= "001000";
941                 RE           <= RE_NONE;
942                 WE           <= WE_BYTE;
943                 addr_bus_sel <= '1';
944                 last_cycle   <= '0';
945             elsif cycleCnt = 2 then
946                 alu_IN      <= ALU_IN_DATAIMM;
947                 alu_gen      <= ALU_GEN_AND;
948                 dau_controls <= "0100010000";
949             end if;
950
951     elsif std_match(IR, OR_RM_RN) then -- OR Rm,Rn
952         reg_ALU_IN      <= ("000" & nreg & "1");
953         reg_MAU         <= (nreg & mreg & "00000000");
954         alu_IN          <= ALU_IN_REGREG;
955         alu_gen         <= ALU_GEN_OR;
956
957     elsif std_match(IR, OR_IMM_IMM_R0) then -- OR #imm,R0
958         reg_ALU_IN      <= REG_IN_R0_REGFILE;
959         alu_IN          <= ALU_IN_REGIMM;
960         alu_gen         <= ALU_GEN_OR;
961
962     elsif std_match(IR, OR_B_IMM_IMM_AT_R0_GBR) then -- OR.B #imm,@(R0,GBR)
963         if cycleCnt = 0 then
964             alu_IN      <= ALU_IN_DATAIMM;
965             alu_gen      <= ALU_GEN_OR;
966             dau_controls <= "0100010000";
967             pau_controls <= "001000";
968             RE           <= RE_BYTE;
969             addr_bus_sel <= '1';
970             last_cycle   <= '0';
971             elsif cycleCnt = 1 then
972                 alu_IN      <= ALU_IN_DATAIMM;
973                 alu_gen      <= ALU_GEN_OR;
974                 alu_opa_hold <= '1';
975                 dau_controls <= "0100010000";
976                 pau_controls <= "001000";
977                 RE           <= RE_NONE;
978                 WE           <= WE_BYTE;
979                 addr_bus_sel <= '1';
980                 last_cycle   <= '0';
981             elsif cycleCnt = 2 then
982                 alu_IN      <= ALU_IN_DATAIMM;

```

```

983         alu_gen          <= ALU_GEN_OR;
984         dau_controls      <= "0100010000";
985     end if;
986
987     elsif std_match(IR, XOR_RM_RN) then -- XOR Rm,Rn
988         reg_ALU_IN        <= ("000" & nreg & "1");
989         reg_MAU           <= (nreg & mreg & "0000000000");
990         alu_IN            <= ALU_IN_REGREG;
991         alu_gen           <= ALU_GEN_XOR;
992
993     elsif std_match(IR, XOR_IMM_IMM_R0) then -- XOR #imm,R0
994         reg_ALU_IN        <= REG_IN_R0_REGFILE;
995         alu_IN            <= ALU_IN_REGIMM;
996         alu_gen           <= ALU_GEN_XOR;
997
998     elsif std_match(IR, XOR_B_IMM_IMM_AT_R0_GBR) then -- XOR.B #imm,@(R0,GBR)
999         if cycleCnt = 0 then
1000             alu_IN        <= ALU_IN_DATAIMM;
1001             alu_gen        <= ALU_GEN_XOR;
1002             dau_controls    <= "0100010000";
1003             pau_controls    <= "001000";
1004             RE              <= RE_BYTE;
1005             addr_bus_sel    <= '1';
1006             last_cycle      <= '0';
1007         elsif cycleCnt = 1 then
1008             alu_IN        <= ALU_IN_DATAIMM;
1009             alu_gen        <= ALU_GEN_XOR;
1010             alu_opa_hold    <= '1';
1011             dau_controls    <= "0100010000";
1012             pau_controls    <= "001000";
1013             RE              <= RE_NONE;
1014             WE              <= WE_BYTE;
1015             addr_bus_sel    <= '1';
1016             last_cycle      <= '0';
1017         elsif cycleCnt = 2 then
1018             alu_IN        <= ALU_IN_DATAIMM;
1019             alu_gen        <= ALU_GEN_XOR;
1020             dau_controls    <= "0100010000";
1021         end if;
1022
1023     elsif std_match(IR, NOT_RM_RN) then -- NOT Rm,Rn
1024         reg_ALU_IN        <= ("000" & nreg & "1");
1025         reg_MAU           <= (mreg & "00000000000000");
1026         alu_IN            <= ALU_IN_REGREG;
1027         alu_gen           <= ALU_GEN_NOT;
1028
1029     elsif std_match(IR, TST_RM_RN) then -- TST Rm,Rn
1030         reg_MAU           <= (nreg & mreg & "0000000000");
1031         alu_IN            <= ALU_IN_REGREG;
1032         alu_gen           <= ALU_GEN_AND;
1033         alu_sh2           <= "00000000011";
1034         tbit_write        <= '1';
1035
1036     elsif std_match(IR, TST_IMM_IMM_R0) then -- TST #imm,R0
1037         alu_IN            <= ALU_IN_REGIMM;
1038         alu_gen           <= ALU_GEN_AND;
1039         alu_sh2           <= "00000000011";
1040         tbit_write        <= '1';
1041
1042     elsif std_match(IR, TST_B_IMM_IMM_AT_R0_GBR) then -- TST.B #imm,@(R0,GBR)
1043         if cycleCnt = 0 then
1044             alu_IN        <= ALU_IN_DATAIMM;
1045             alu_gen        <= ALU_GEN_AND;
1046             alu_sh2        <= "00000000011";
1047             dau_controls    <= "0100010000";
1048             pau_controls    <= "001000";
1049             RE              <= RE_BYTE;

```

```

1050         addr_bus_sel      <= '1';
1051         last_cycle         <= '0';
1052     elsif cycleCnt = 1 then
1053         alu_IN              <= ALU_IN_DATAIMM;
1054         alu_gen             <= ALU_GEN_AND;
1055         alu_opa_hold        <= '1';
1056         alu_sh2             <= "0000000011";
1057         dau_controls        <= "0100010000";
1058         tbit_write          <= '1';
1059     end if;
1060
1061     elsif std_match(IR, TAS_B_AT_RN) then -- TAS.B @Rn
1062         if cycleCnt = 0 then
1063             reg_MAU          <= ("000000000000" & nreg);
1064             alu_IN           <= "0-";
1065             alu_gen          <= ALU_GEN_OR;
1066             alu_sh2          <= "00000001111";
1067             pau_controls     <= "001000";
1068             RE               <= RE_BYTE;
1069             addr_bus_sel     <= '1';
1070             last_cycle       <= '0';
1071         elsif cycleCnt = 1 then
1072             reg_MAU          <= ("000000000000" & nreg);
1073             alu_IN           <= "0-";
1074             alu_gen          <= ALU_GEN_OR;
1075             alu_opa_hold     <= '1';
1076             alu_opb_bit7    <= '1';
1077             alu_sh2          <= "00000001111";
1078             pau_controls     <= "001000";
1079             RE               <= RE_NONE;
1080             WE               <= WE_BYTE;
1081             addr_bus_sel     <= '1';
1082             tbit_write       <= '1';
1083             last_cycle       <= '0';
1084         elsif cycleCnt = 2 then
1085             alu_IN           <= "0-";
1086             alu_gen          <= ALU_GEN_OR;
1087             alu_sh2          <= "00000001111";
1088         end if;
1089
1090     elsif std_match(IR, ROTL_RN) then -- ROTL Rn
1091         reg_ALU_IN          <= ("000" & nreg & "1");
1092         reg_MAU             <= (nreg & "000000000000");
1093         alu_IN              <= ALU_IN_REGREG;
1094         alu_gen              <= ALU_GEN_ROTCL;
1095         alu_sh2             <= "00000000001";
1096         tbit_write          <= '1';
1097
1098     elsif std_match(IR, ROTR_RN) then -- ROTR Rn
1099         reg_ALU_IN          <= ("000" & nreg & "1");
1100         reg_MAU             <= (nreg & "000000000000");
1101         alu_IN              <= ALU_IN_REGREG;
1102         alu_gen              <= ALU_GEN_ROTCL;
1103         alu_sh2             <= "00000000001";
1104         tbit_write          <= '1';
1105
1106     elsif std_match(IR, ROTCL_RN) then -- ROTCL Rn
1107         reg_ALU_IN          <= ("000" & nreg & "1");
1108         reg_MAU             <= (nreg & "000000000000");
1109         alu_IN              <= ALU_IN_REGREG;
1110         alu_gen              <= ALU_GEN_ROTCL;
1111         alu_sh2             <= "00000000001";
1112         tbit_write          <= '1';
1113
1114     elsif std_match(IR, ROTCR_RN) then -- ROTCR Rn
1115         reg_ALU_IN          <= ("000" & nreg & "1");
1116         reg_MAU             <= (nreg & "000000000000");

```

```

1117         alu_IN          <= ALU_IN_REGREG;
1118         alu_gen          <= ALU_GEN_ROTCL;
1119         alu_sh2          <= "0000000001";
1120         tbit_write       <= '1';
1121
1122     elsif std_match(IR, SHLL_RN) then -- SHLL Rn
1123         reg_ALU_IN        <= ("000" & nreg & "1");
1124         reg_MAU           <= (nreg & "000000000000");
1125         alu_IN            <= ALU_IN_REGREG;
1126         alu_gen           <= ALU_GEN_SHLL1;
1127         alu_sh2           <= "0000000001";
1128         tbit_write       <= '1';
1129
1130     elsif std_match(IR, SHLR_RN) then -- SHLR Rn
1131         reg_ALU_IN        <= ("000" & nreg & "1");
1132         reg_MAU           <= (nreg & "000000000000");
1133         alu_IN            <= ALU_IN_REGREG;
1134         alu_gen           <= ALU_GEN_SHLR1_L;
1135         alu_sh2           <= "0000000001";
1136         tbit_write       <= '1';
1137
1138     elsif std_match(IR, SHAL_RN) then -- SHAL Rn
1139         reg_ALU_IN        <= ("000" & nreg & "1");
1140         reg_MAU           <= (nreg & "000000000000");
1141         alu_IN            <= ALU_IN_REGREG;
1142         alu_gen           <= ALU_GEN_SHLL1;
1143         alu_sh2           <= "0000000001";
1144         tbit_write       <= '1';
1145
1146     elsif std_match(IR, SHAR_RN) then -- SHAR Rn
1147         reg_ALU_IN        <= ("000" & nreg & "1");
1148         reg_MAU           <= (nreg & "000000000000");
1149         alu_IN            <= ALU_IN_REGREG;
1150         alu_gen           <= ALU_GEN_SHLR1_A;
1151         alu_sh2           <= "0000000001";
1152         tbit_write       <= '1';
1153
1154     elsif std_match(IR, SHLL2_RN) then -- SHLL2 Rn
1155         reg_ALU_IN        <= ("000" & nreg & "1");
1156         reg_MAU           <= (nreg & "000000000000");
1157         alu_IN            <= ALU_IN_REGREG;
1158         alu_gen           <= ALU_GEN_PASS;
1159         alu_sh2           <= "0000010000";
1160
1161     elsif std_match(IR, SHLL8_RN) then -- SHLL8 Rn
1162         reg_ALU_IN        <= ("000" & nreg & "1");
1163         reg_MAU           <= (nreg & "000000000000");
1164         alu_IN            <= ALU_IN_REGREG;
1165         alu_gen           <= ALU_GEN_PASS;
1166         alu_sh2           <= "0100010000";
1167
1168     elsif std_match(IR, SHLL16_RN) then -- SHLL16 Rn
1169         reg_ALU_IN        <= ("000" & nreg & "1");
1170         reg_MAU           <= (nreg & "000000000000");
1171         alu_IN            <= ALU_IN_REGREG;
1172         alu_gen           <= ALU_GEN_PASS;
1173         alu_sh2           <= "1000010000";
1174
1175     elsif std_match(IR, SHLR2_RN) then -- SHLR2 Rn
1176         reg_ALU_IN        <= ("000" & nreg & "1");
1177         reg_MAU           <= (nreg & "000000000000");
1178         alu_IN            <= ALU_IN_REGREG;
1179         alu_gen           <= ALU_GEN_SHLR_N;
1180         alu_sh2           <= "0000010000";
1181
1182     elsif std_match(IR, SHLR8_RN) then -- SHLR8 Rn
1183         reg_ALU_IN        <= ("000" & nreg & "1");

```

```

1184     reg_MAU          <= (nreg & "000000000000");
1185     alu_IN           <= ALU_IN_REGREG;
1186     alu_gen          <= ALU_GEN_SHLR_N;
1187     alu_sh2          <= "0100010000";
1188
1189     elsif std_match(IR, SHLR16_RN) then -- SHLR16 Rn
1190         reg_ALU_IN    <= ("000" & nreg & "1");
1191         reg_MAU        <= (nreg & "000000000000");
1192         alu_IN         <= ALU_IN_REGREG;
1193         alu_gen         <= ALU_GEN_SHLR_N;
1194         alu_sh2        <= "10000010000";
1195
1196     elsif std_match(IR, BT_LABEL) then -- BT label
1197         if Tbit = '1' then
1198             alu_IN      <= ALU_IN_DATAREG;
1199             alu_gen      <= ALU_GEN_PASS;
1200             pau_controls <= PAU_BRANCH_DISP;
1201         else
1202             alu_IN      <= ALU_IN_DATAREG;
1203             alu_gen      <= ALU_GEN_PASS;
1204         end if;
1205
1206     elsif std_match(IR, BF_LABEL) then -- BF label
1207         if Tbit = '0' then
1208             alu_IN      <= ALU_IN_DATAREG;
1209             alu_gen      <= ALU_GEN_PASS;
1210             pau_controls <= PAU_BRANCH_DISP;
1211         else
1212             alu_IN      <= ALU_IN_DATAREG;
1213             alu_gen      <= ALU_GEN_PASS;
1214         end if;
1215
1216     elsif std_match(IR, BT_S_LABEL) then -- BT/S label (delayed conditional)
1217         if Tbit = '1' then
1218             pau_controls <= PAU_BRANCH_DISP;
1219             delay_slot   <= '1';
1220         end if;
1221
1222     elsif std_match(IR, BF_S_LABEL) then -- BF/S label (delayed conditional)
1223         if Tbit = '0' then
1224             pau_controls <= PAU_BRANCH_DISP;
1225             delay_slot   <= '1';
1226         end if;
1227
1228     elsif std_match(IR, BRA_LABEL) then -- BRA label (delayed)
1229         imm_sel    <= "10";
1230         pau_controls <= PAU_BRANCH_DISP;
1231         delay_slot <= '1';
1232
1233     elsif std_match(IR, BRAF_RM) then -- BRAF Rm (delayed)
1234         reg_MAU    <= (IR(11 downto 8) & "000000000000");
1235         pau_controls <= PAU_BRANCH_REG;
1236         delay_slot  <= '1';
1237
1238     elsif std_match(IR, BSR_LABEL) then -- BSR label (delayed)
1239         imm_sel    <= "10";
1240         pau_controls <= PAU_BSR;
1241         delay_slot  <= '1';
1242
1243     elsif std_match(IR, BSRF_RM) then -- BSRF Rm (delayed)
1244         reg_MAU    <= (IR(11 downto 8) & "000000000000");
1245         pau_controls <= PAU_BSRF;
1246         delay_slot  <= '1';
1247
1248     elsif std_match(IR, JMP_AT_RM) then -- JMP @Rm (delayed)
1249         reg_MAU    <= (IR(11 downto 8) & "000000000000");
1250         pau_controls <= PAU_JMP;

```

```

1251         delay_slot          <= '1';
1252
1253     elsif std_match(IR, JSR_AT_RM) then -- JSR @Rm (delayed)
1254         reg_MAU              <= (IR(11 downto 8) & "000000000000");
1255         pau_controls         <= PAU_JSR;
1256         delay_slot          <= '1';
1257
1258     elsif std_match(IR, RTS) then -- RTS (delayed)
1259         pau_controls         <= PAU_RTS;
1260         delay_slot          <= '1';
1261
1262     elsif std_match(IR, CLRT) then -- CLRT
1263         alu_IN               <= ALU_IN_DATAREG;
1264         alu_gen              <= ALU_GEN_NONE;
1265         alu_sh2              <= "0000001101";
1266         tbit_write          <= '1';
1267
1268     elsif std_match(IR, SETT) then -- SETT
1269         alu_IN               <= ALU_IN_DATAREG;
1270         alu_gen              <= ALU_GEN_NONE;
1271         alu_sh2              <= "0000001110";
1272         tbit_write          <= '1';
1273
1274     elsif std_match(IR, NOP) then -- NOP
1275         alu_IN               <= ALU_IN_DATAREG;
1276         alu_gen              <= ALU_GEN_PASS;
1277
1278     elsif std_match(IR, LDC_RM_GBR) then -- LDC Rm,GBR
1279         reg_MAU              <= ("000000000000" & IR(11 downto 8));
1280         alu_IN               <= ALU_IN_DATAREG;
1281         alu_gen              <= ALU_GEN_PASS;
1282         dau_write            <= DAU_WRITE_GBR;
1283
1284     elsif std_match(IR, LDC_RM_VBR) then -- LDC Rm,VBR
1285         reg_MAU              <= ("000000000000" & IR(11 downto 8));
1286         alu_IN               <= ALU_IN_DATAREG;
1287         alu_gen              <= ALU_GEN_PASS;
1288         dau_write            <= DAU_WRITE_VBR;
1289
1290     elsif std_match(IR, LDC_L_AT_RM_PLUS_GBR) then -- LDC.L @Rm+,GBR
1291         if cycleCnt = 0 then
1292             reg_MAU          <= ("00000000" & IR(11 downto 8) & "1" & IR(11 downto 8));
1293             alu_IN           <= ALU_IN_DATAREG;
1294             alu_gen          <= ALU_GEN_PASS;
1295             dau_controls     <= "0000100100";
1296             dau_write        <= DAU_WRITE_GBR;
1297             dau_load_data    <= '1';
1298             pau_controls     <= "001000";
1299             addr_bus_sel     <= '1';
1300             last_cycle       <= '0';
1301         elsif cycleCnt = 1 then
1302             reg_MAU          <= ("00000000" & IR(11 downto 8) & "0" & IR(11 downto 8));
1303             alu_IN           <= ALU_IN_DATAREG;
1304             alu_gen          <= ALU_GEN_PASS;
1305             dau_controls     <= "0000100100";
1306         end if;
1307
1308     elsif std_match(IR, LDC_L_AT_RM_PLUS_VBR) then -- LDC.L @Rm+,VBR
1309         if cycleCnt = 0 then
1310             reg_MAU          <= ("00000000" & IR(11 downto 8) & "1" & IR(11 downto 8));
1311             alu_IN           <= ALU_IN_DATAREG;
1312             alu_gen          <= ALU_GEN_PASS;
1313             dau_controls     <= "0000100100";
1314             dau_write        <= DAU_WRITE_VBR;
1315             dau_load_data    <= '1';
1316             pau_controls     <= "001000";
1317             addr_bus_sel     <= '1';

```

```

1318         last_cycle           <= '0';
1319     elsif cycleCnt = 1 then
1320         reg_MAU               <= ("00000000" & IR(11 downto 8) & "0" & IR(11 downto 8));
1321         alu_IN                <= ALU_IN_DATAREG;
1322         alu_gen               <= ALU_GEN_PASS;
1323         dau_controls          <= "0000100100";
1324     end if;
1325
1326     elsif std_match(IR, LDC_L_AT_RM_PLUS_SR) then -- LDC.L @Rm+,SR
1327         if cycleCnt = 0 then
1328             reg_MAU           <= ("00000000" & IR(11 downto 8) & "1" & IR(11 downto 8));
1329             alu_IN            <= ALU_IN_DATAREG;
1330             alu_gen           <= ALU_GEN_PASS;
1331             dau_controls      <= "0000100100";
1332             pau_controls      <= "001000";
1333             addr_bus_sel     <= '1';
1334             last_cycle        <= '0';
1335         elsif cycleCnt = 1 then
1336             reg_MAU           <= ("00000000" & IR(11 downto 8) & "0" & IR(11 downto 8));
1337             alu_IN            <= ALU_IN_DATAREG;
1338             alu_gen           <= ALU_GEN_PASS;
1339             alu_sh2           <= "00000001010";
1340             tbit_write        <= '1';
1341         end if;
1342
1343     elsif std_match(IR, STC_SR_RN) then -- STC SR,Rn
1344         reg_ALU_IN           <= ("101" & nreg & "1");
1345
1346     elsif std_match(IR, STC_GBR_RN) then -- STC GBR,Rn
1347         reg_ALU_IN           <= ("100" & nreg & "1");
1348         alu_IN               <= ALU_IN_DATAREG;
1349         alu_gen              <= ALU_GEN_PASS;
1350         dau_controls         <= "0100100000";
1351         dau_out_sel          <= "01";
1352
1353     elsif std_match(IR, STC_VBR_RN) then -- STC VBR,Rn
1354         reg_ALU_IN           <= ("100" & nreg & "1");
1355         alu_IN               <= ALU_IN_DATAREG;
1356         alu_gen              <= ALU_GEN_PASS;
1357         dau_controls         <= "1000100000";
1358         dau_out_sel          <= "10";
1359
1360     elsif std_match(IR, STC_L_SR_AT_MINUSRN) then -- STC.L SR,@-Rn
1361         if cycleCnt = 0 then
1362             reg_MAU           <= ("00000000" & nreg & "1" & nreg);
1363             alu_IN            <= ALU_IN_DATAREG;
1364             alu_gen           <= ALU_GEN_PASS;
1365             dau_controls      <= "0000101101";
1366             pau_controls      <= "001000";
1367             RE                <= RE_NONE;
1368             WE                <= WE_LONG;
1369             data_bus_sel      <= "10";
1370             addr_bus_sel     <= '1';
1371             last_cycle        <= '0';
1372         elsif cycleCnt = 1 then
1373             reg_MAU           <= ("00000000" & nreg & "0" & nreg);
1374             alu_IN            <= ALU_IN_DATAREG;
1375             alu_gen           <= ALU_GEN_PASS;
1376             dau_controls      <= "0000101101";
1377         end if;
1378
1379     elsif std_match(IR, STC_L_GBR_AT_MINUSRN) then -- STC.L GBR,@-Rn
1380         if cycleCnt = 0 then
1381             reg_MAU           <= ("00000000" & nreg & "1" & nreg);
1382             alu_IN            <= ALU_IN_DATAREG;
1383             alu_gen           <= ALU_GEN_PASS;
1384             dau_controls      <= "0000101101";

```

```

1385     dau_out_sel      <= "01";
1386     pau_controls     <= "001000";
1387     RE               <= RE_NONE;
1388     WE               <= WE_LONG;
1389     data_bus_sel     <= "01";
1390     addr_bus_sel     <= '1';
1391     last_cycle       <= '0';
1392     elsif cycleCnt = 1 then
1393         reg_MAU       <= ("00000000" & nreg & "0" & nreg);
1394         alu_IN        <= ALU_IN_DATAREG;
1395         alu_gen       <= ALU_GEN_PASS;
1396         dau_controls  <= "0000101101";
1397         dau_out_sel   <= "01";
1398     end if;
1399
1400     elsif std_match(IR, STC_L_VBR_AT_MINUSRN) then -- STC.L VBR,@-Rn
1401         if cycleCnt = 0 then
1402             reg_MAU       <= ("00000000" & nreg & "1" & nreg);
1403             alu_IN        <= ALU_IN_DATAREG;
1404             alu_gen       <= ALU_GEN_PASS;
1405             dau_controls  <= "0000101101";
1406             dau_out_sel   <= "10";
1407             pau_controls  <= "001000";
1408             RE           <= RE_NONE;
1409             WE           <= WE_LONG;
1410             data_bus_sel <= "01";
1411             addr_bus_sel <= '1';
1412             last_cycle   <= '0';
1413             elsif cycleCnt = 1 then
1414                 reg_MAU       <= ("00000000" & nreg & "0" & nreg);
1415                 alu_IN        <= ALU_IN_DATAREG;
1416                 alu_gen       <= ALU_GEN_PASS;
1417                 dau_controls  <= "0000101101";
1418                 dau_out_sel   <= "10";
1419             end if;
1420
1421         elsif std_match(IR, LDS_RM_PR) then -- LDS Rm,PR
1422             if cycleCnt = 0 then
1423                 reg_MAU       <= (IR(11 downto 8) & "000000000000");
1424                 alu_IN        <= ALU_IN_REGREG;
1425                 alu_gen       <= ALU_GEN_PASS;
1426                 pau_controls  <= "001010";
1427                 RE           <= RE_NONE;
1428                 last_cycle   <= '0';
1429             elsif cycleCnt = 1 then
1430                 reg_MAU       <= (IR(11 downto 8) & "000000000000");
1431                 alu_IN        <= ALU_IN_REGREG;
1432                 alu_gen       <= ALU_GEN_PASS;
1433             end if;
1434
1435         elsif std_match(IR, LDS_L_AT_RM_PLUS_PR) then -- LDS.L @Rm+,PR
1436             if cycleCnt = 0 then
1437                 reg_MAU       <= ("00000000" & IR(11 downto 8) & "1" & IR(11 downto 8));
1438                 alu_IN        <= ALU_IN_DATAREG;
1439                 alu_gen       <= ALU_GEN_PASS;
1440                 dau_controls  <= "0000100100";
1441                 pau_controls  <= PAU_LDS_PR;
1442                 addr_bus_sel <= '1';
1443                 last_cycle   <= '0';
1444             elsif cycleCnt = 1 then
1445                 reg_MAU       <= ("00000000" & IR(11 downto 8) & "0" & IR(11 downto 8));
1446                 alu_IN        <= ALU_IN_DATAREG;
1447                 alu_gen       <= ALU_GEN_PASS;
1448                 dau_controls  <= "0000100100";
1449             end if;
1450
1451         elsif std_match(IR, STS_PR_RN) then -- STS PR,Rn

```

```

1452         reg_ALU_IN          <= ("011" & nreg & "1");
1453         alu_IN               <= ALU_IN_DATAREG;
1454         alu_gen              <= ALU_GEN_PASS;
1455
1456     elsif std_match(IR, STS_L_PR_AT_MINUSRN) then -- STS.L PR,@-Rn
1457         if cycleCnt = 0 then
1458             reg_MAU           <= ("00000000" & nreg & "1" & nreg);
1459             alu_IN            <= ALU_IN_DATAREG;
1460             alu_gen           <= ALU_GEN_PASS;
1461             dau_controls       <= "0000101101";
1462             pau_controls       <= "001000";
1463             RE                 <= RE_NONE;
1464             WE                 <= WE_LONG;
1465             data_bus_sel       <= "11";
1466             addr_bus_sel       <= '1';
1467             last_cycle         <= '0';
1468         else
1469             reg_MAU           <= ("00000000" & nreg & "0" & nreg);
1470             alu_IN            <= ALU_IN_DATAREG;
1471             alu_gen           <= ALU_GEN_PASS;
1472             dau_controls       <= "0000101101";
1473         end if;
1474
1475     end if;
1476
1477     -- Suppress delay_slot during delay slot execution so a branch in the
1478     -- delay slot cannot trigger nested delay slots TODO
1479     if delay_slot_reg = '1' then
1480         delay_slot <= '0';
1481     end if;
1482
1483 end process;
1484
1485
1486
1487
1488 end structural;
1489
1490

```